 # Topics :

* CDK Basics
  - Definitions
  - Constructs
  - Assets
* CDK Workshop
* Advanced CDK
  - Testing
  - Aspects
  - Best Practices

 # What is CDK ?

- AWS Cloud Development Kit (AWS CDK)
- Open Source Software
- Enables writing imperative code to generate declarative cloudformation templates
- Javascript, Typescript, Python, Java, C# and Go

 # What is CloudFormation ?

- AWS service that provisions aws resources in stacks.
- A stack is a collection of aws resources.
- Stack templates are in yaml / json format.
- Templates are declarative code.
- Cloudformation creates aws resources from templates.

 # Layers of a CDK App :

* App : no cloudformation equivalent
  - Stack : cloudformation stack
    * Nested stack : cloudformation nested stack
    * Construct : 1 or more cloudformation resources
* Can have multiple apps
* Apps can have multiple stacks
* The output of CDK is cloudformation

 # What is a CDK App ?

- App is a special rool construct.
- Orchestrates the lifecycle of the stacks and resources within it.
- App Lifecycle :
  * construct -> prepare -> validate -> synthesize -> deploy

 # What is a CDK Stack ?

- A CDK Stack becomes a cloudformation stack.
- Multiple stacks can be within an app.
- Stacks within the same app can share resources.
- CDK nested stacks generate cloudformation nested stacks
  * Stacks with stacks as resources.

 # What are CDK Constructs ?

* Level 0 : Basic Resource (no type)
* Level 1 : Cloudformation Resource (1:1)
  - These are pre fixed with cfn in the cdk api
* Level 2 : Improved L1 Constructs
  - Provided by the cdk team
  - Include helper methods and sensible defaults
* Level 3 : Combinations of constructs

Note : most frequently you'll use L2/L3 constructs

 # Level 1 Construct Example :

- starts with cfn
- 1:1 with cloudformation
- no defaults
- no helper functions

 # Level 2 Construct Example :

- no prefix
- extend L1 constructs with sensible defaults
- includes helper methods

 # Level 3 Construct Example :

- cdkpatterns.com
- aws solutions constructs
- construct hub : constructs.dev
- inner / open source constructs

 # Synthesis

- Running `cdk synth` generates the cloudformation template.
- It traverses the App tree and invokes the synthesize method on all constructs.
- Generates unique ids for cloudformation.

 # Assets

- Assets are files bundled into cdk apps.
  * lambda handler code
  * docker images
- Assets can represent any artifact that the app needs to operate.
- When you synthesize or deploy a cdk app, these typically end up in cdk.out

 # Bootstrapping

- Running `cdk bootstrap` deploys a cdk toolkit cloudformation stack.
- Create an s3 bucket to store assets in an aws account.
- Required for assets and cloudformation templates > 50kb.

 # Deploy

- App is initialized into an app tree.
- Methods of all constructs are called in series :
  * prepare -> validate -> synthesize
- Deployment artifacts are uploaded to cdk toolkit and cloudformation deployment begins.

 # CDK Workshop Speedrun

- cdkworkshop.com

1. New Project
2. Hello CDK
3. Writing Constructs
4. Using Construct Libraries
5. Testing Constructs
6. Advanced CDK

 # Testing

- Fine grained assertions
- Snapshot tests
- Validation tests
- Integration tests
  * Using aws custom resources to test as part of the cloudformation deployment
  * Custom resources enable you to write custom provisioning logic in templates
  * CDK has a provider framework for interacting with cloudformation custom resources.

 # Aspects

- Aspects are a way to apply an operation to all constructs in a given scope.

 # Best Practices

1. Organization
- Team of cdk experts :
  * set standards
  * training and mentoring developers
- Deploy to multiple aws accounts :
  * use ci/cd tools like cdk pipelines for deploying beyond dev accounts

2. Coding
- Keep it simple stupid
- Use the aws well architected framework
- One app per repo
- Infrastructure and runtime code should be in the same place

3. Construct
- Model with constructs, deploy with stacks
- Configure with properties and methods
- Unit test
- Don't change the logical id
- Constructs are not enough for compliance

4. Application
- Make decisions at synthesis
- Specify as few names as possible
- Define removel policies and log retention
- Use multiple stacks
- Commit cdk.context.json
- Let cdk manage roles / security groups
- Model all production stages in code
- Measure everything

Resources :

- How to create a three tier serverless application on freecodecamp.
- Openapi specs from cdk stack without deploying first
- Serverless stack framework (SST)
  a. serverless-stack.com
  b. extension of cdk